Django JWT Authentication API

Technical Documentation — Self-Hosted, Containerized REST API with JWT-Based Authentication

1. Project Overview

This document describes a Django and Django REST Framework (DRF) application implementing JWT-based authentication with refresh token rotation, token blacklisting, and role-based access control. The application was developed to demonstrate a production-representative authentication pattern in a controlled, self-hosted environment.

The system is containerized using Docker and Docker Compose, and was built and validated on a local development machine. It comprises a custom user model, an administrator/user role system, and a set of REST endpoints for user account management.

Objectives

- Implement JWT access and refresh token issuance and validation
- Store tokens securely using HttpOnly cookies rather than client-accessible storage (e.g., localStorage)
- Implement refresh token rotation with blacklisting of superseded or reused tokens
- Enforce role-based access control (administrator vs. standard user)
- Containerize the application stack using Docker for consistent local deployment

2. Tech Stack

Component	Technology
Backend framework	Django 6.0.7
API layer	Django REST Framework (DRF) 3.17.1
Frontend framework	React (served via Vite dev server, Node.js 22)
Authentication	djangorestframework-simplejwt 5.5.1 (JWT) + custom cookie auth class
Token blacklisting	rest_framework_simplejwt.token_blacklist
Database	PostgreSQL 17 (via psycopg2-binary)
CORS handling	django-cors-headers
Static files	WhiteNoise
WSGI server	Gunicorn
Containerization	Docker + Docker Compose (db, backend, frontend services)
Env config	python-dotenv

3. Authentication Flow (JWT + Cookies)

Rather than storing the JWT in localStorage or transmitting it manually via headers, the application issues both the access and refresh tokens as HttpOnly cookies. This renders the tokens inaccessible to client-side JavaScript, reducing exposure to XSS-based token theft.

3.1 Login

- The client submits a username and password to POST /login/
- Django's authenticate() function verifies the credentials against the custom user model
- Upon success, SimpleJWT issues an access token and a refresh token
- Both tokens are set as HttpOnly cookies (access_token, refresh_token) and are not exposed to client-side JavaScript
- The response body returns basic profile information (id, username, email, role) but excludes the raw tokens

3.2 Authenticated Requests

A custom authentication class, CookieJWTAuthentication (which extends SimpleJWT's JWTAuthentication), first checks for a standard Authorization: Bearer header. If absent, it falls back to reading the access_token cookie. This allows the API to support both cookie-based browser clients and header-based API clients.

3.3 Token Expiry, Rotation & Blacklisting

Setting	Value
ACCESS_TOKEN_LIFETIME	10 minutes
REFRESH_TOKEN_LIFETIME	3 days
ROTATE_REFRESH_TOKENS	True — a new refresh token is issued on each refresh
BLACKLIST_AFTER_ROTATION	True — the prior refresh token is blacklisted once rotated
AUTH_COOKIE_HTTP_ONLY	True
AUTH_COOKIE_SAMESITE	Lax

Each time a refresh token is used, it is invalidated and replaced with a new one. Consequently, a leaked or reused refresh token cannot be used to generate further access tokens. This rotation-and-blacklisting behavior is the core security mechanism the application was built to demonstrate.

3.4 Logout

POST /logout/ clears both cookies from the client. A recommended enhancement, noted in Section 9, is to also explicitly blacklist the current refresh token on the server side at logout.

4. Roles & Permissions

The custom user model (app1.Login, configured as AUTH_USER_MODEL) extends Django's AbstractUser and includes a role field used for role-based access control via two custom DRF permission classes:

- IsAdmin — grants access only if request.user.role == "admin"
- IsUser — grants access only if request.user.role == "user"

These classes are combined using DRF's | (OR) operator on views such as GetUserById and UpdateUser, allowing either an administrator or the resource owner to access them. List and create operations are restricted to administrators only.

5. API Endpoints

Method	Endpoint	Purpose	Access
POST	/login/	Authenticates the user and issues JWT access and refresh tokens as HttpOnly cookies	Public
POST	/logout/	Clears the access and refresh token cookies	Public
POST	/create/	Creates a new user account	Admin only
GET	/users/	Lists all user accounts	Admin only
GET	/userdata/<id>/	Retrieves a single user's account data	Owner (User) or Admin
PUT / PATCH	/update/<id>/	Performs a full or partial update of a user record	Owner (User) or Admin

Note: the URL configuration currently registers GetUser under /users/ twice — once under the administrator section and once under the user section. This is a duplicate route in the current urls.py and does not introduce a second behavior; it is noted here as a cleanup item.

6. Docker Setup

The application runs as three containers orchestrated with Docker Compose: a PostgreSQL database, the Django/Gunicorn backend, and the React/Vite frontend.

Dockerfile

```
FROM python:3.13-slim

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

WORKDIR /app
COPY requirements.txt .

RUN pip install --upgrade pip
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
EXPOSE 8000

CMD ["gunicorn", "p1.wsgi:application", "--bind", "0.0.0.0:8000"]
```

Building & Running (Backend Container)

```
# Build the image
docker build -t django-jwt-auth .

# Run the container (maps port 8000, loads environment variables from .env)
docker run --env-file .env -p 8000:8000 django-jwt-auth
```

The application connects to PostgreSQL using environment variables (DB_NAME, DB_USER, DB_PASSWORD, DB_HOST, DB_PORT) loaded via python-dotenv. A .env file, or an equivalent linked PostgreSQL container / Docker Compose configuration, is required alongside the application container for the database connection to succeed.

Frontend Container

A separate Dockerfile defines the frontend, running a Node.js/Vite development server. This configuration is consistent with the backend's CORS_ALLOWED_ORIGINS and CSRF_TRUSTED_ORIGINS settings, both of which specify http://localhost:5173.

```
FROM node:22-alpine

WORKDIR /app
COPY package*.json ./

RUN npm install
COPY . .

EXPOSE 5173
CMD ["npm","run","dev","--","--host"]
```

Building and running the frontend container:

```
# Build the image
docker build -t django-jwt-frontend .

# Run the container (Vite development server on port 5173)
docker run -p 5173:5173 django-jwt-frontend
```

Because the API sets cookies as HttpOnly with SameSite=Lax, the frontend and backend must run on origins the browser treats as compatible for cookies to be transmitted. This is why the exact localhost:5173 origin is explicitly allowed rather than a wildcard origin.

Docker Compose (Orchestrating All Three Services)

A docker-compose.yml file at the project root coordinates the database, backend, and frontend services so the full stack can be started with a single command.

Service	Details
db	postgres:17 — reads POSTGRES_DB/USER/PASSWORD from .env, persists data to a named volume (postgres_data), exposed on host port 5432
backend	Built from ./p1; runs Gunicorn on port 8000; mounts the source code as a volume for live editing; reads configuration from .env; depends on db
frontend	Built from ./frontend; runs the Vite development server on port 5173; mounts source code as a volume (node_modules retained within the container); depends on backend

```
# Start the full stack (db + backend + frontend)
docker compose up --build

# Run in the background
docker compose up -d --build

# Stop all services
docker compose down
```

depends_on controls container start order (db → backend → frontend) but does not wait for PostgreSQL to be ready to accept connections. In the current configuration, this is mitigated by Gunicorn/Django's connection retry behavior, or by restarting Docker Compose if the backend starts before PostgreSQL completes initialization. A health check on the db service would make this startup sequence more robust (see Section 9).

7. Configuration Highlights (settings.py)

- Custom user model: AUTH_USER_MODEL = "app1.Login"
- SECRET_KEY, database credentials, and DEBUG are all loaded from environment variables via python-dotenv (.env) rather than hardcoded
- Default DRF authentication class: app1.authentication.CookieJWTAuthentication
- Default DRF permission class: IsAuthenticated; endpoints are restricted by default, with explicit AllowAny applied only to /login/ and /logout/
- CORS is restricted to http://localhost:5173 (the Vite/React development server) with credentials enabled, allowing cookies to be transmitted cross-origin during local development

8. Admin Panel

Django's built-in admin site (django.contrib.admin) is enabled, providing direct database-level visibility into user records for local testing and debugging. This is distinct from the role-based "admin" application role used within the API itself.

9. Recommended Future Improvements

- Add two-factor authentication (2FA) as an additional verification step during login, e.g. via a time-based one-time password (TOTP) app or email/SMS code
- Add OAuth-based social login (e.g. "Sign in with Google") as an alternative to username/password authentication, using a library such as django-allauth or authlib
- Explicitly blacklist the refresh token on logout, rather than only clearing cookies
- Remove the duplicate /users/ route registration in urls.py
- Add a health check or wait-for-it strategy so the backend does not start before PostgreSQL is ready to accept connections